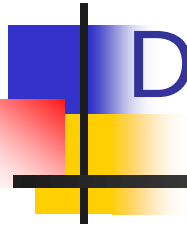


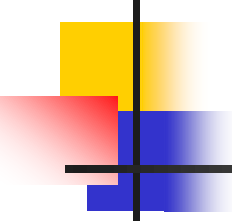
CS501

Introductory Computation for Scientists



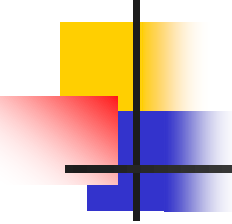
Dr. Ash Pahwa

Lesson 5: Python Functions



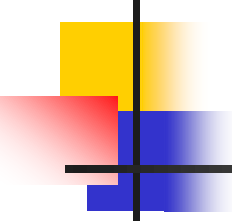
Outline

- Purpose of a Function
- Defining and Calling Function
- Default and Keyword Argument
- Scope of Variables
- Recursive Functions



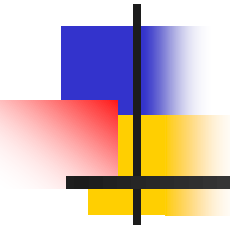
Purpose of a Function

- Any repeated computation can be written as function
- Advantages of a function
 - No need to write the same code again and again instead write a function and call it repeatedly
 - Functions can be called repeatedly with different arguments
 - Functions allow us to write better software
 - Less bugs
 - Faster execution: Functions can be optimized for better performance



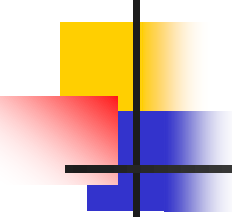
Built-in Functions in Python

- `Sum(<array>)`
- `Min(<list>)`
- `Abs(<number>)`



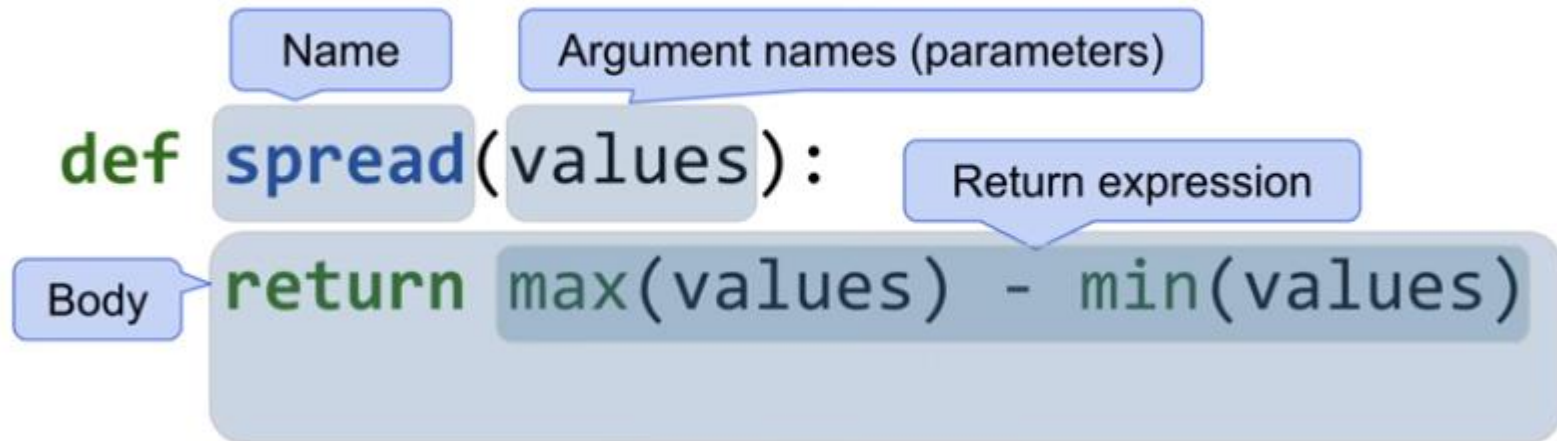
Defining and Calling Functions

2.7.1



Def Statement

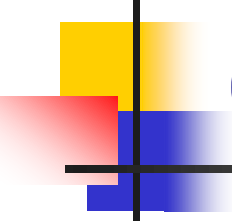
User-defined functions give names to blocks of code



```
def spread(values):  
    return max(values) - min(values)
```

The diagram illustrates the components of a Python `def` statement using callouts:

- Name:** Points to the function name `spread`.
- Argument names (parameters):** Points to the parameter `values` in the parentheses.
- Return expression:** Points to the expression `max(values) - min(values)` following the colon.
- Body:** Points to the `return` statement and its associated expression.



Define a Function

Call a Function

```
In [1]: def double(x):  
        return 2 * x
```

```
In [2]: double(3)
```

```
Out[2]: 6
```

```
In [3]: double(-4)
```

```
Out[3]: -8
```

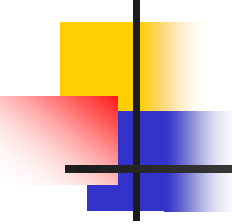
```
In [4]: y=5  
        double(y/4)
```

```
Out[4]: 2.5
```

```
In [5]: z = double(y)
```

```
In [6]: z
```

```
Out[6]: 10
```



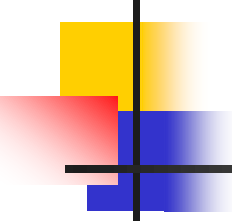
Define a Function

Call a Function

```
In [1]: def square(x):  
        return x**2
```

```
In [2]: print(square(2))  
4
```

```
In [3]: print(square(12))  
144
```

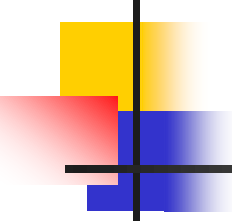
Quadratic Equation

- Example
 - Quadratic Equation: $ax^2 + bx + c = 0$
 - Closed Form Solution
 - $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
 - $x^2 - x - 6 = 0$
 - $(x - 3) * (x + 2) = 0$
 - $x = 3 \quad x = -2$

```
In [10]: import math
def roots(a,b,c):
    d = b**2 - 4*a*c
    r1 = ( -b + math.sqrt(d))/(2*a)
    r2 = ( -b - math.sqrt(d))/(2*a)
    return r1, r2
```

```
In [11]: print(roots(1,-1,-6))

(3.0, -2.0)
```



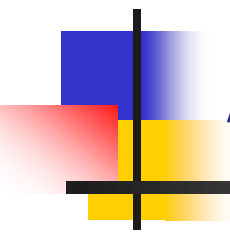
Docstrings

```
In [12]: #Docstrings
def roots(a,b,c):
    ''' Computes the roots of a quadratic equation '''
    d = b**2 - 4*a*c
    r1 = ( -b + math.sqrt(d))/(2*a)
    r2 = ( -b - math.sqrt(d))/(2*a)
    return r1, r2
```

```
In [13]: help(roots)

Help on function roots in module __main__:

roots(a, b, c)
    Computes the roots of a quadratic equation
```



Default and Keyword Arguments

2.7.2

Keyword Arguments

Section 2.7.2 Default and Keyword arguments

```
In [32]: def roots(a,b,c):  
        ''' Computes the roots of a quadratic equation '''  
        d = b**2 - 4*a*c  
        r1 = ( -b + math.sqrt(d))/(2*a)  
        r2 = ( -b - math.sqrt(d))/(2*a)  
        return r1, r2
```

```
In [33]: roots ( a = 1., c = -6., b = -1.)
```

```
Out[33]: (3.0, -2.0)
```

```
In [34]: roots ( 1., c = -6., b = -1.)
```

```
Out[34]: (3.0, -2.0)
```

```
In [35]: roots ( 1., -6., b = -1.)
```

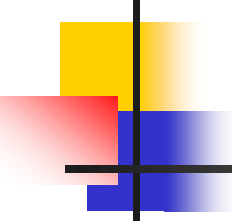
```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-35-44831d7b3144> in <module>()  
----> 1 roots ( 1., -6., b = -1.)
```

```
TypeError: roots() got multiple values for argument 'b'
```

```
In [36]: roots ( b = -1., 1., -6.)
```

```
File "<ipython-input-36-581adca0c563>", line 1  
    roots ( b = -1., 1., -6.)  
              ^
```

```
SyntaxError: positional argument follows keyword argument
```



Discussion Question

What does this function do? What kind of input does it take? What output will it give? What's a reasonable name?

```
def f(s):  
    return np.round(s / sum(s) * 100, 2)
```

Arguments

```
In [68]: import numpy as np

counts = np.array([1,2,3])
total = sum(counts)
np.round(counts/total * 100, 2)
```

```
Out[68]: array([ 16.67,  33.33,  50.  ])
```

```
In [69]: def percents(counts):
...     This is a comment line
...     Converts an array into percentages
...     total = sum(counts)
...     return np.round(counts/total * 100, 2)
```

```
In [71]: percents(counts)
```

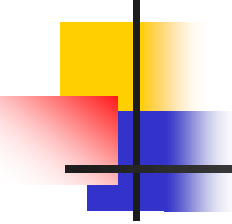
```
Out[71]: array([ 16.67,  33.33,  50.  ])
```

```
In [72]: percents(2)
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-72-c13170656183> in <module>()
----> 1 percents(2)

<ipython-input-69-eda0ad5b886f> in percents(counts)
      4     Converts an array into percentages
      5     ...
----> 6     total = sum(counts)
      7     return np.round(counts/total * 100, 2)

TypeError: 'int' object is not iterable
```



Error

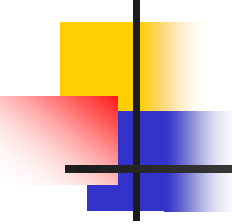
If proper Arguments are not Fed to the Function

```
In [71]: percents(counts)
```

```
Out[71]: array([ 16.67,  33.33,  50.  ])
```

```
In [72]: percents(2)
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-72-c13170656183> in <module>()  
----> 1 percents(2)  
  
<ipython-input-69-eda0ad5b886f> in percents(counts)  
      4     Converts an array into percentages  
      5     '''  
----> 6     total = sum(counts)  
      7     return np.round(counts/total * 100, 2)  
  
TypeError: 'int' object is not iterable
```



More Arguments

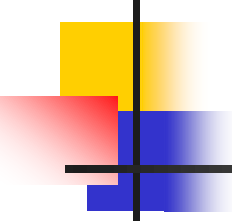
```
In [23]: def percents(counts, decimal_places):  
        ...  
        This is a comment line  
        Converts an array into percentages  
        ...  
        total = sum(counts)  
        return np.round(counts/total * 100, decimal_places)
```

```
In [24]: percents(counts,2)
```

```
Out[24]: array([ 16.67,  33.33,  50.  ])
```

```
In [25]: percents(counts,3)
```

```
Out[25]: array([ 16.667,  33.333,  50.  ])
```

Default Argument

```
In [26]: def percents(counts, decimal_places=2):  
        ...  
        This is a comment line  
        Converts an array into percentages  
        ...  
        total = sum(counts)  
        return np.round(counts/total * 100, decimal_places)
```

```
In [27]: percents(counts)
```

```
Out[27]: array([ 16.67,  33.33,  50.  ])
```

```
In [28]: percents(counts,3)
```

```
Out[28]: array([ 16.667,  33.333,  50.  ])
```

Calling Function

Arguments – by name instead of position

Getting help of a function

```
In [29]: percents(counts,decimal_places=4)
```

```
Out[29]: array([ 16.6667,  33.3333,  50.    ])
```

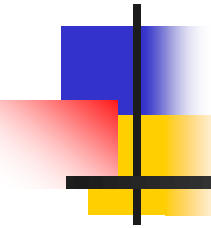
```
In [31]: help(percents)
```

```
Help on function percents in module __main__:
```

```
percents(counts, decimal_places=2)
```

```
    This is a comment line
```

```
    Converts an array into percentages
```



Scope of Variables

2.7.3

Function Variables are Local Not Available Outside Function

Global Variables are available everywhere

```
In [43]: def func():  
         a = 5  
         print(a,b)
```

```
In [45]: b = 6  
         func()  
5 6
```

Function Variables are Local Not Available Outside Function

Global Variables are available everywhere

```
In [46]: def func():  
         a = 5  
         print(a)
```

```
In [47]: a=6  
         func()  
  
         5
```

```
In [48]: print(a)  
  
         6
```

Global Variables

Global variables

```
In [58]: def func1():  
         print(x)  
  
         def func2():  
             x += 1
```

```
In [59]: x = 4  
         func1()
```

4

```
In [60]: func2()
```

```
-----  
UnboundLocalError                                Traceback (most recent call last)  
<ipython-input-60-dff3c14b0386> in <module>()  
----> 1 func2()  
  
<ipython-input-58-eee91aad4833> in func2()  
      3  
      4 def func2():  
----> 5     x += 1  
  
UnboundLocalError: local variable 'x' referenced before assignment
```

```
In [61]: def func1():  
         print(x)
```

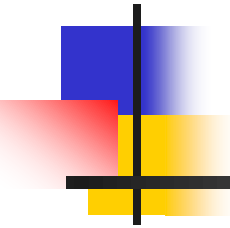
```
         def func2():  
             global x  
             x += 1
```

```
In [62]: x = 4  
         func1()
```

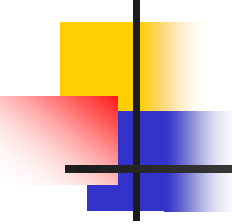
4

```
In [63]: func2()  
         x
```

Out[63]: 5



Recursive Functions



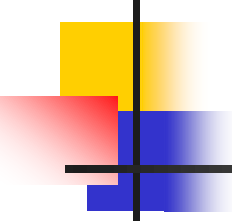
Recursive Functions

Recursive Functions

```
In [64]: def factorial(n):  
         if n == 1:  
             return 1  
         return n * factorial(n-1)
```

```
In [65]: factorial(5)
```

```
Out[65]: 120
```

Summary

- Purpose of a Function
- Defining and Calling Function
- Default and Keyword Argument
- Scope of Variables
- Recursive Functions

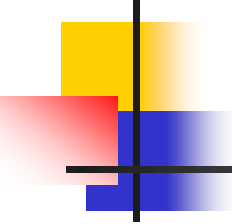
CS501

Introductory Computation for Scientists



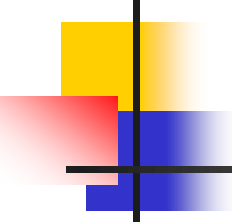
Dr. Ash Pahwa

Lesson 12.1: Sampling and Simulation-1



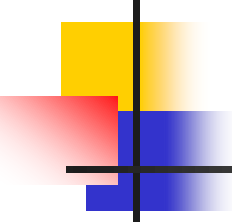
Outline

- Statistics
- Probability
 - Probability Simulation in Python
- Random Sampling
 - Sampling in Python



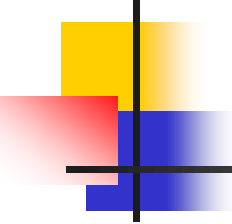
What is Statistics?

- Statistics is the Science of
 - Collecting
 - Organizing
 - Summarizing
 - Analyzing
- Information to draw conclusions
- Statistics is about providing a measure of confidence in any decision



Foundation of Data Science

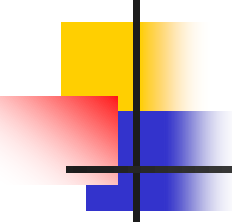
- Foundation of Data Science and Machine Learning
 - Statistics
 - Statistics allows us to take correct decisions using data
 - Machines take correct decision using data
- Machine Learning Algorithms
 - Use statistics as the foundation



Modeling Methods

All Modeling Methods use Statistics

#	Modeling Methods
1	Linear & Polynomial Regression
2	Logistic Regression
3	Discriminant Analysis
4	K Nearest Neighbor
5	Decision and Regression Trees
6	Naïve Bayes
7	Neural Networks
8	Clustering
9	Principal Component Analysis
10	Support Vector Machines
11	ARIMA : Time Series



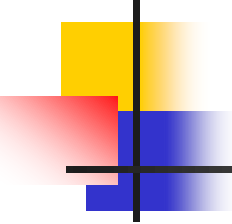
Goal of this Course: CS 501

- Goal of this course is NOT to study statistics
 - But to see how we can implement statistical functions in Python

Few Words to Summarize a Discipline

- Economics
 - Money
- Accounting
 - How much?
- Biology
 - Life
- Psychology
 - Why we think what we think?
- History
 - What, where and when?
- **Statistics**
 - **Variation**

- Data vary because
 - We don't see everything
 - We can't measure everything
- How to make decision
 - Based on the imperfect data
- Statistics helps us
 - Making correct decisions

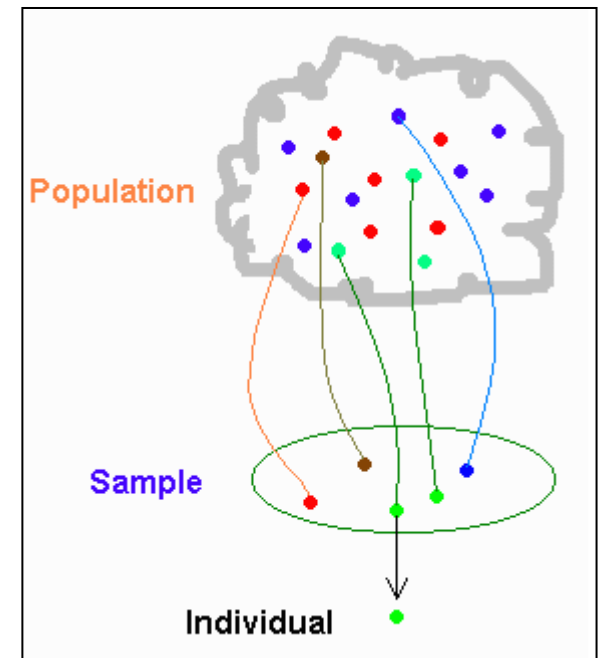


Applications of Statistics: Examples

- Anecdotal claims
 - Cellular telephones cause brain cancer?
 - Smoking causes cancer?
- Collect and analyze data using statistics
 - Conclusion
 - No link between cell phone usage and brain cancer
 - Yes, smoking causes lung cancer

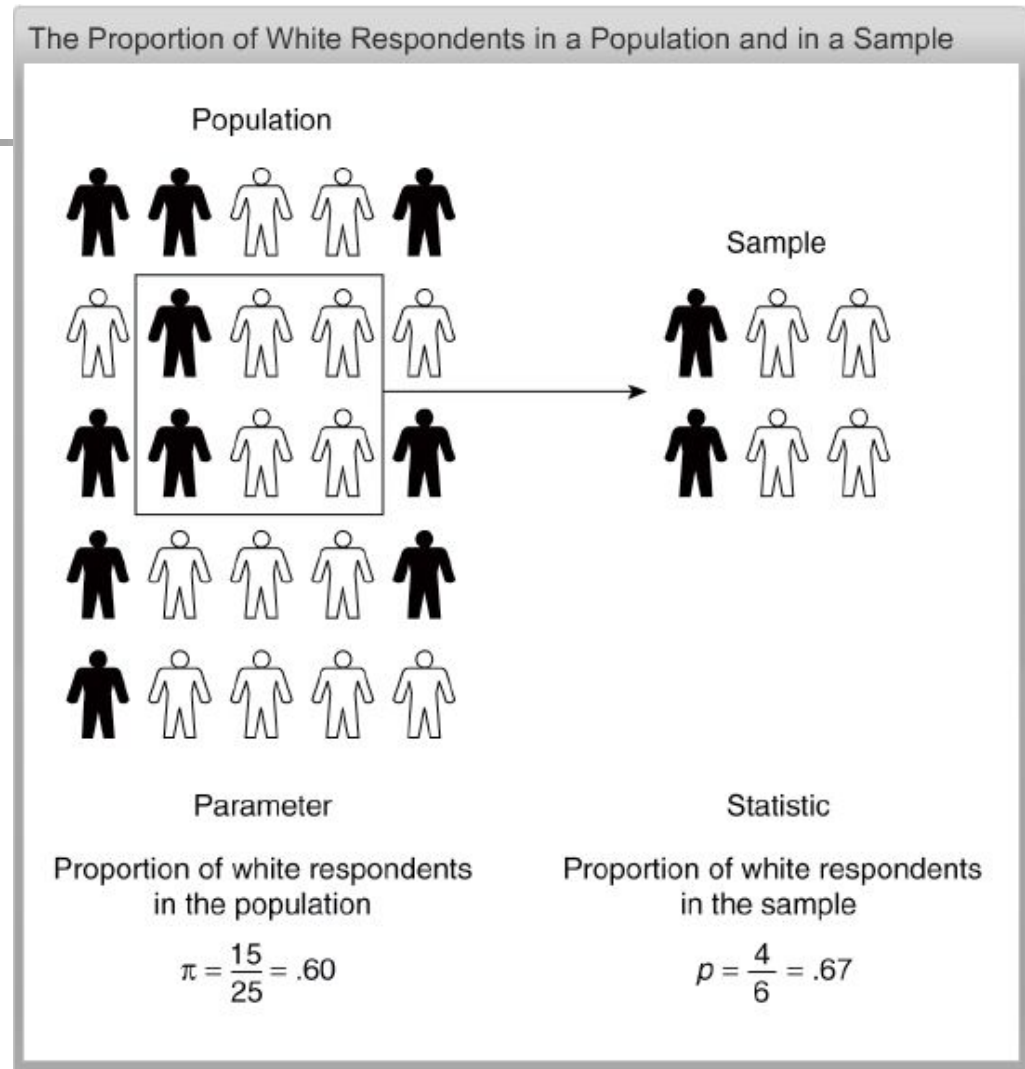
Basic Definition

- Population
 - Entire group of individuals or objects to be studied is called population
- Sample
 - Subset of population
 - That is being studied
- Individual
 - Person or object that is a part of the population being studied



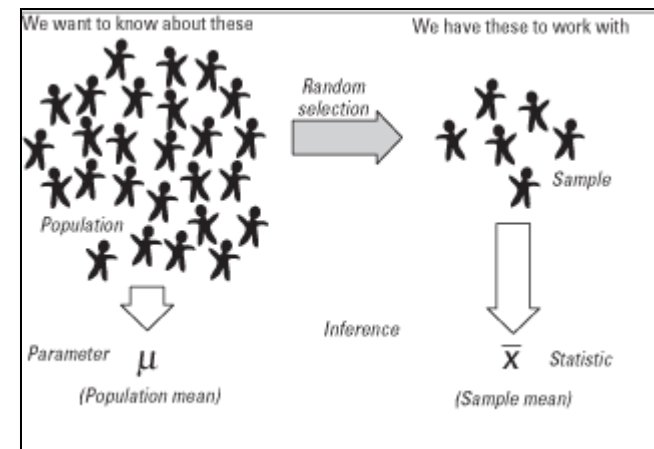
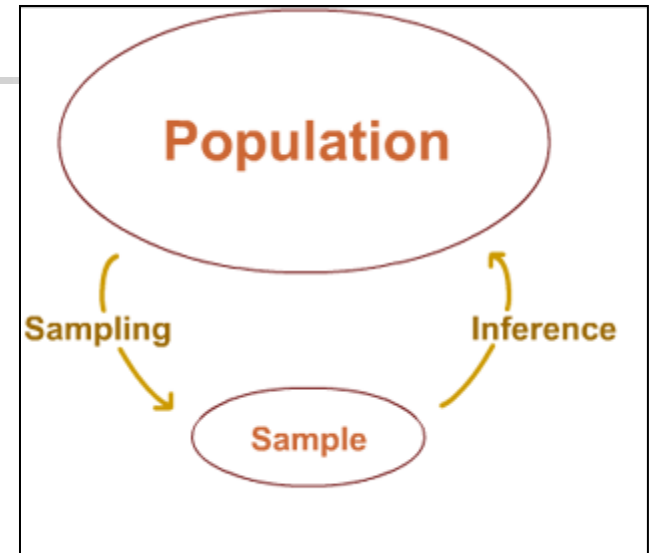
Statistics and Parameter

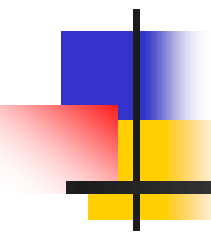
- **Statistics:**
 - Numerical summary of a sample
- **Parameter:**
 - Numerical summary of a population



Inference Statistics

- Inferential Statistics uses methods that take a result from a sample,
 - extend it to the population,
 - and measure the reliability of the result

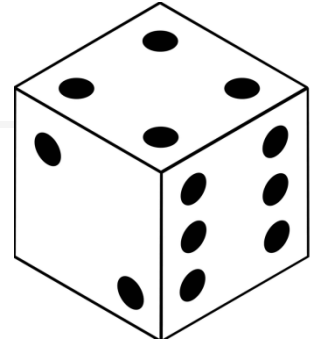


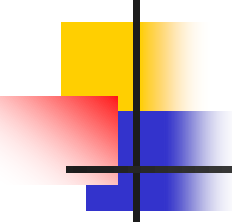


Probability

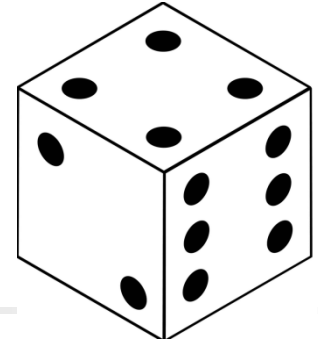
Die and Dice

- Singular: Die
- The **Plural** of "Die" Is "**Dice**"



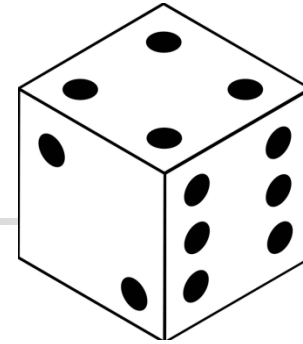


Probability

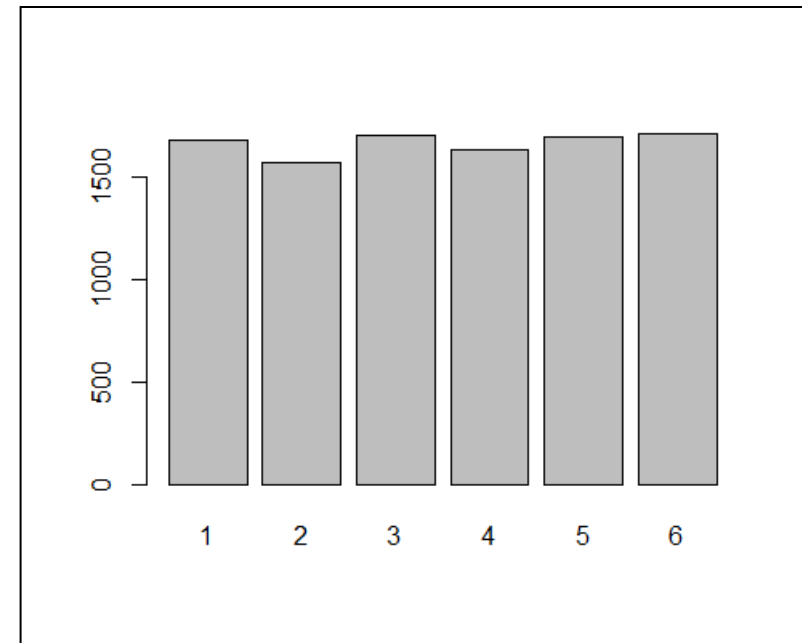


- If you roll a die, what is the probability that you will get a face with 5 dots
 - 1/6
- *Probability of occurrence = $\frac{X}{T}$*
- *X = number of ways in which an event can occur*
- *T = Total number of possible outcomes*

Distribution



- Toss die 10,000 times
- X-axis
 - Sample space
 - 1,2,3,4,5,6
- Y-axis
 - Frequency of each number



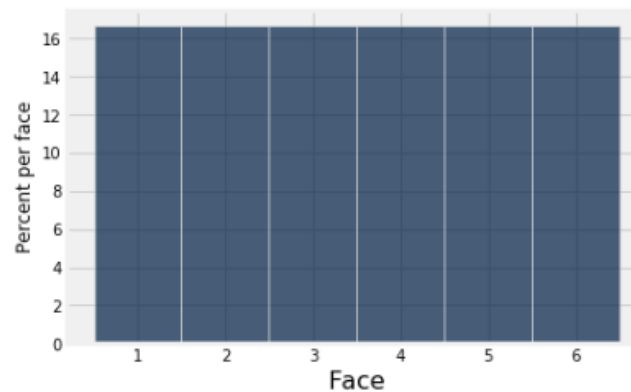
Die Histogram

```
In [9]: die = Table().with_column('face',np.arange(6)+1)  
die
```

```
Out[9]: face  
1  
2  
3  
4  
5  
6
```

```
In [10]: def face_hist(t):  
          t.hist('face',bins=np.arange(0.5,7,1), unit='face')  
          plots.xlabel('Face')
```

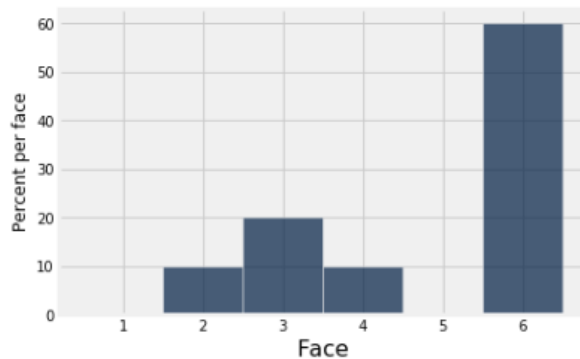
```
In [11]: face_hist(die)
```



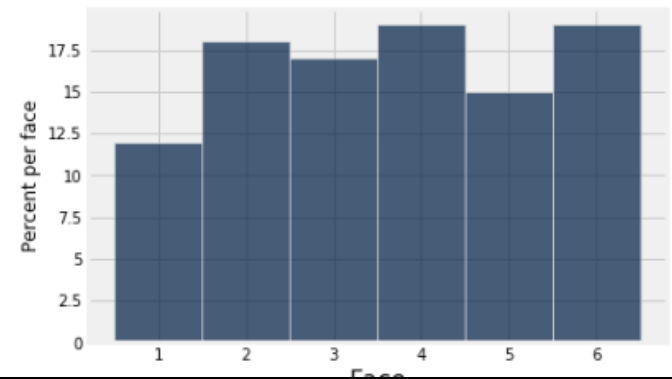
Toss a Die: Histogram

10, 100, 10,000 time

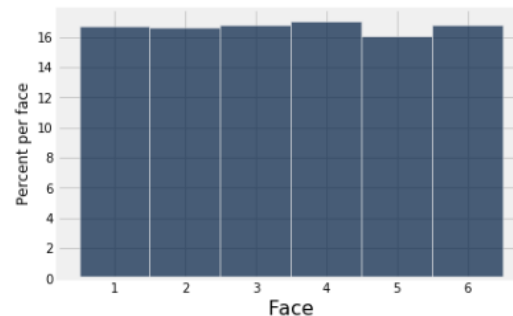
```
In [13]: die.sample(5)  
face_hist(die.sample(10))
```



```
In [14]: face_hist(die.sample(100))
```



```
In [15]: face_hist(die.sample(10000))
```





Numpy Random Function

Counting Random Samples

```
In [9]: two_groups = make_array('treatment', 'control')  
        np.random.choice(two_groups)
```

```
Out[9]: 'treatment'
```

```
In [10]: np.random.choice(two_groups, 10)
```

```
Out[10]: array(['treatment', 'treatment', 'treatment', 'treatment', 'treatment',  
               'control', 'treatment', 'control', 'treatment', 'control'],  
              dtype='<U9')
```

```
In [11]: # Counting  
        outcomes = np.random.choice(two_groups, 10)  
        outcomes == 'treatment'
```

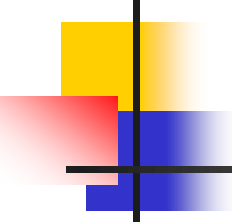
```
Out[11]: array([ True,  True, False,  True,  True, False,  True,  True,  True,  True], dtype=bool)
```

```
In [12]: sum(outcomes == 'treatment')
```

```
Out[12]: 8
```

```
In [13]: np.count_nonzero(outcomes == 'treatment')
```

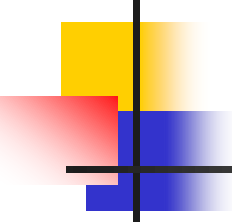
```
Out[13]: 8
```



Discussion Question

A population has 100 people, including Mo and Jo.
We sample two people at random without replacement.

(a) $P(\text{both Mo and Jo are in the sample})$



Discussion Question

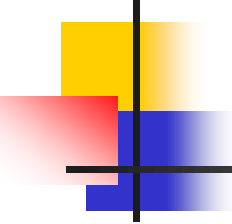
A population has 100 people, including Mo and Jo.
We sample two people at random without replacement.

(a) $P(\text{both Mo and Jo are in the sample})$

$= P(\text{first Mo, then Jo}) + P(\text{first Jo, then Mo})$

$= (1/100) * (1/99) + (1/100) * (1/99) = 0.0002$

(b) $P(\text{neither Mo nor Jo is in the sample})$



Discussion Question

A population has 100 people, including Mo and Jo.
We sample two people at random without replacement.

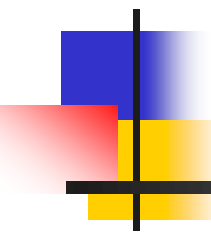
(a) $P(\text{both Mo and Jo are in the sample})$

$= P(\text{first Mo, then Jo}) + P(\text{first Jo, then Mo})$

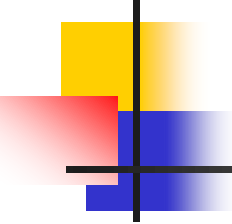
$$= (1/100) * (1/99) + (1/100) * (1/99) = 0.0002$$

(b) $P(\text{neither Mo nor Jo is in the sample})$

$$= (98/100) * (97/99) = 0.9602$$

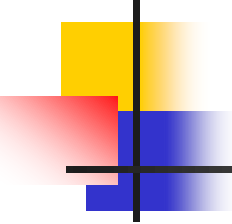


Random Sampling



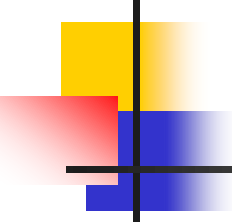
Sampling

- Deterministic sample:
 - Sampling scheme doesn't involve chance.
- Probability sample:
 - Before the sample is drawn, you have to know the selection probability of every group in the population.
 - Not all individuals have to have equal chances of being selected.
- Uniform random sample:
 - Each individual has an equal chance of being selected.
 - A "simple random sample" is uniform & without replacement.



Sample of Convenience

- Example: sample consists of whoever walks by.
- Just because you think you're sampling "at random", doesn't mean you are.
- If you can't figure out ahead of time
 - what's the population, and
 - what's the chance of selection, for each group in the populationthen you don't have a random sample.



Random

```
from datascience import *
import numpy as np
import matplotlib.pyplot as plots
plots.style.use('fivethirtyeight')
%matplotlib inline

import random
```

```
In [5]: for i in range(10):
        print(random.uniform(3,7))
```

```
3.66480196455945
4.798892072203346
5.5419625494767475
3.8982591457491917
6.833624354010206
3.2803562753696665
6.701024515136362
5.082642318974834
3.059968771756169
4.005300354931039
```

```
In [2]: for i in range(10):
        print(random.random())
```

```
0.45874861801098155
0.3390726775678238
0.061042208684796484
0.7448824224982021
0.6432310453116502
0.646664516955829
0.5889976213157156
0.4175780660979237
0.20345026232993235
0.008409885846942577
```

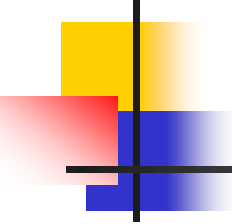
```
In [3]: # Generate a random number between 3 and 7: [3,7)
        #
```

```
# Since width is 4 (7-3 = 4), we multiply with 4
# since the first number is 3, we add 3
```

```
def my_random():
    return 4*random.random() + 3
```

```
In [4]: for i in range(10):
        print(my_random())
```

```
6.148531288340264
4.847857064431148
5.549321233324919
3.417223243197144
4.643795082557738
5.819459819900251
5.805790806329746
4.95061393866157
5.931151964264892
3.276063578741155
```



Random Number Generation from Normally Distributed Data

```
for i in range(10):  
    print(random.normalvariate(0,1))
```

```
0.10462585764737198  
-2.498915951032077  
1.7483600686538057  
0.8578400719356467  
-1.4559006310137799  
1.1632724883635268  
-0.05716743019248963  
-0.720926599669413  
-1.3106214468439428  
-1.128907081371045
```

```
In [7]: for i in range(10):  
        print(random.normalvariate(0,9))
```

```
3.750820938458162  
0.20958862834535052  
16.10333723478306  
-3.691269379205106  
12.092354981662716  
-12.02622169209639  
16.662280009144794  
-0.24647219723304073  
-2.7382733944076607  
-0.11041678241602357
```

```
In [8]: for i in range(10):  
        print(random.normalvariate(0,0.2))
```

```
0.043815604680860314  
0.19264239522500615  
-0.10366072272988786  
0.5293066947369709  
0.03861172450946146  
0.33408422368114615  
-0.15072658150104162  
-0.11577495254532418  
0.042713270640547994  
0.6444796076813278
```

Random Number Generation from Normally Distributed Data

$$\mu = 0, \sigma = 3$$

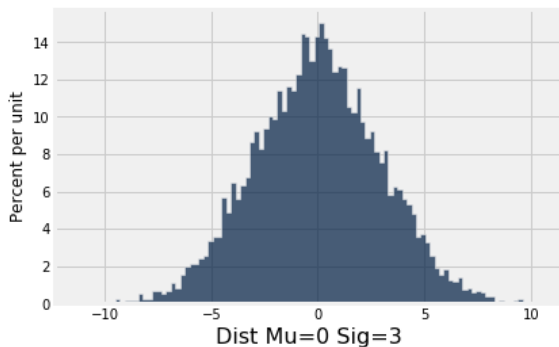
$$\mu = 0, \sigma = 3$$

$$\mu = 0, \sigma = 1$$

```
In [23]: numbers_to_generate = 10000
num_array = make_array()

for i in range(numbers_to_generate):
    num = random.normalvariate(0,3)
    num_array = np.append(num_array, num)

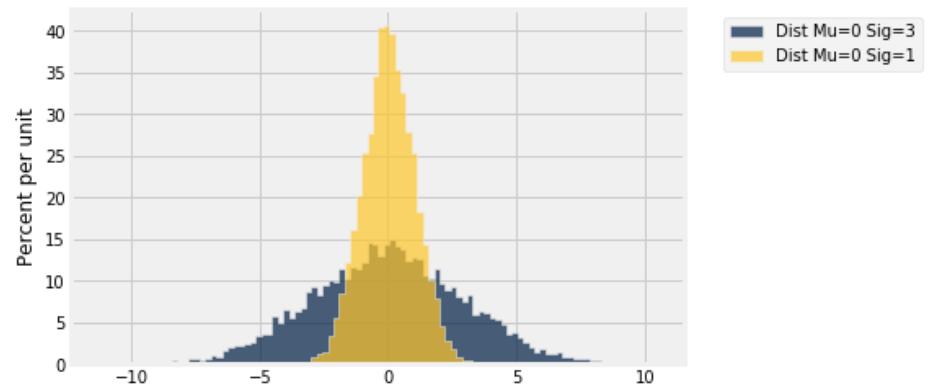
t = Table().with_column('Dist Mu=0 Sig=3', num_array)
t.hist(bins=100)
```



```
In [24]: num_array = make_array()

for i in range(numbers_to_generate):
    num = random.normalvariate(0,1)
    num_array = np.append(num_array, num)

t = t.with_column('Dist Mu=0 Sig=1', num_array)
t.hist(bins=100)
```



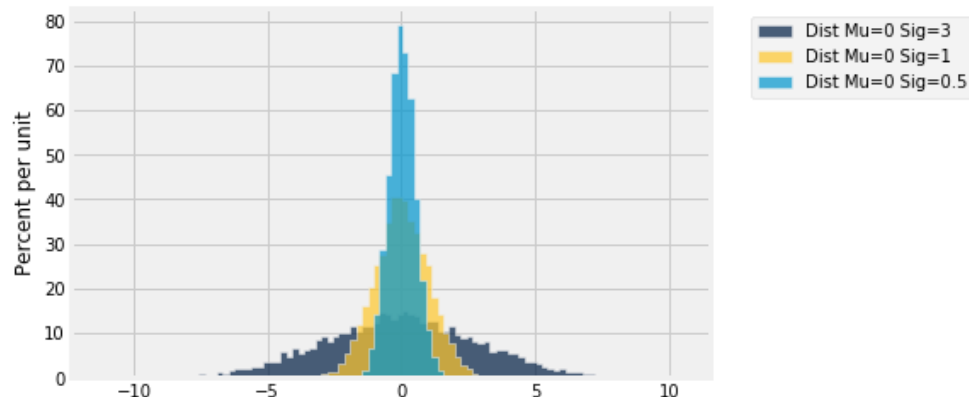
Random Number Generation from Normally Distributed Data

$$\mu = 0, \sigma = 3$$
$$\mu = 0, \sigma = 1$$
$$\mu = 0, \sigma = 0.5$$

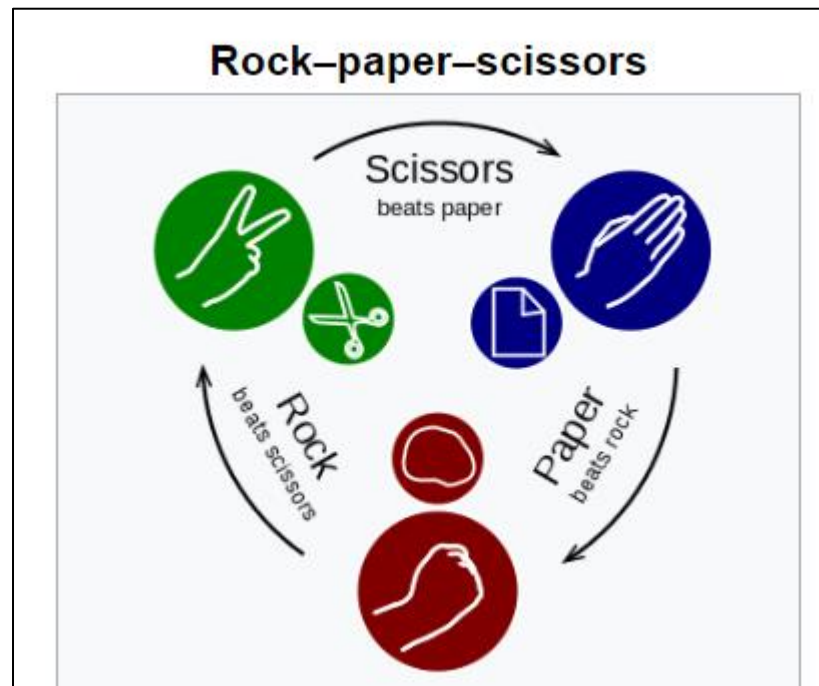
```
In [25]: num_array = make_array()

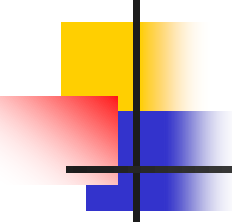
for i in range(numbers_to_generate):
    num = random.normalvariate(0,0.5)
    num_array = np.append(num_array, num)

t = t.with_column('Dist Mu=0 Sig=0.5', num_array)
t.hist(bins=100)
```



Rock, Paper, Scissors





Python Code

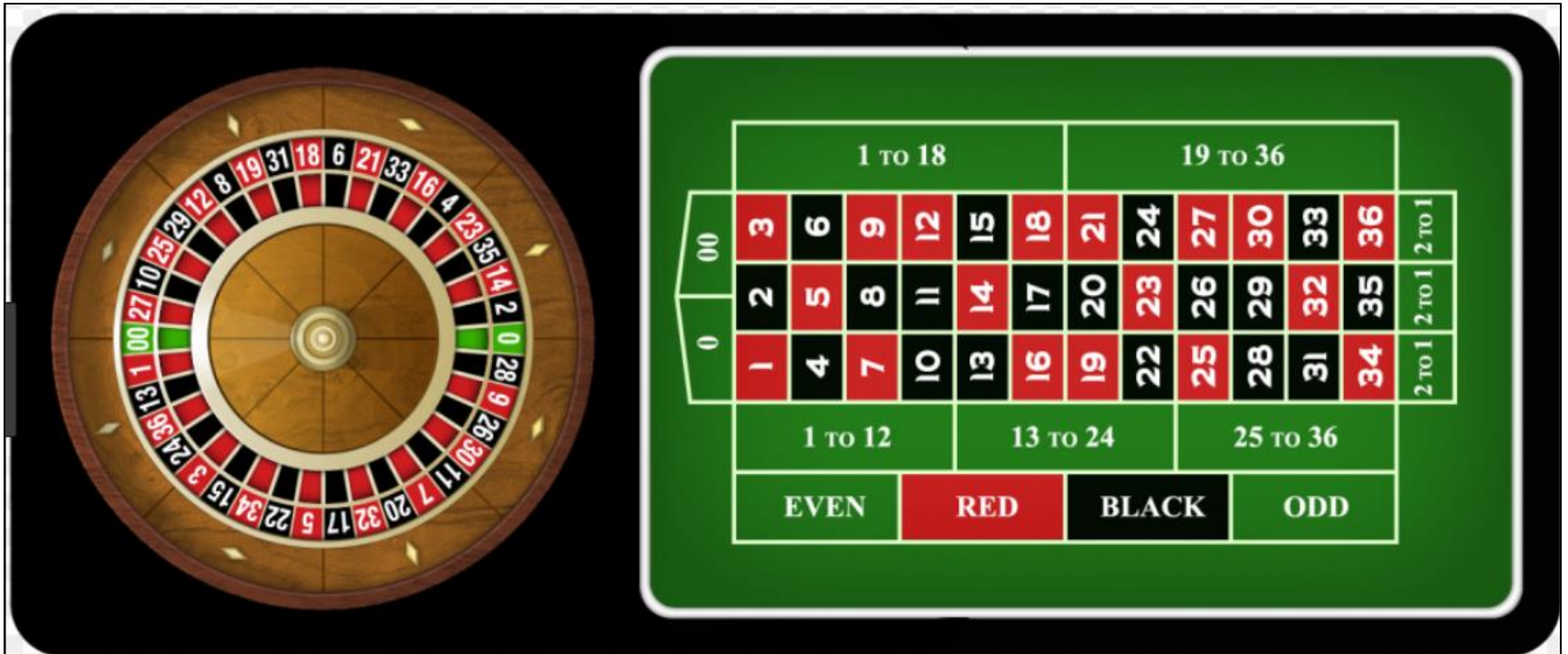
```
In [10]: outcomes = ['rock', 'paper', 'scissors']  
         for i in range(10):  
             print(random.choice(outcomes))
```

```
scissors  
paper  
scissors  
rock  
rock  
paper  
rock  
scissors  
paper  
paper
```

```
In [11]: random.choice(outcomes)
```

```
Out[11]: 'rock'
```

Roulette Table



Simulation: Roulette

In [14]: `# Roulette`

```
colors = ['Red', 'Black', 'Green']  
for i in range(10):  
    print(random.choice(colors))
```

Black
Red
Red
Red
Green
Red
Green
Black
Black
Green

In [15]: `for i in range(10):
 print(random.choices(colors, k = 10))`

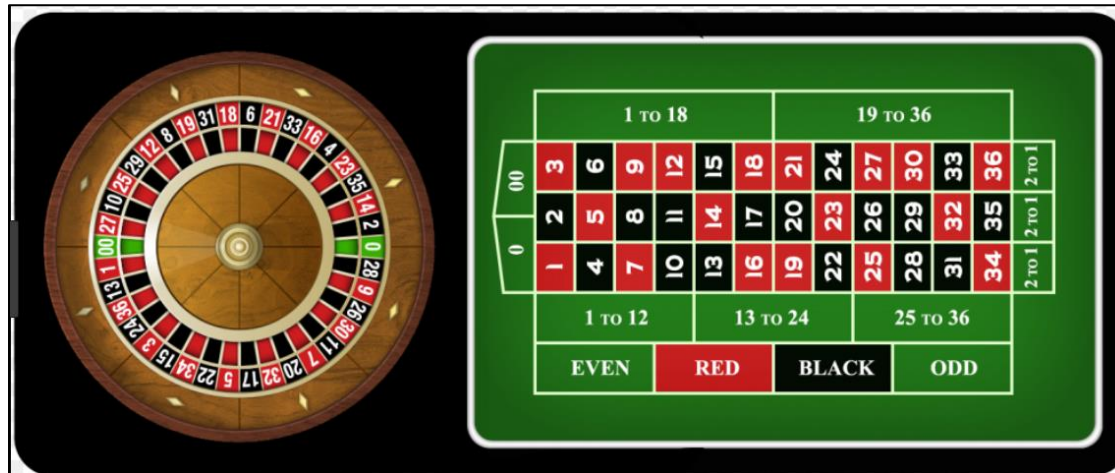
```
['Red', 'Black', 'Red', 'Red', 'Green', 'Red', 'Red', 'Green', 'Black', 'Green']  
['Green', 'Red', 'Black', 'Green', 'Red', 'Black', 'Red', 'Black', 'Red', 'Black']  
['Red', 'Red', 'Red', 'Red', 'Green', 'Green', 'Red', 'Green', 'Red', 'Green']  
['Green', 'Red', 'Red', 'Black', 'Red', 'Black', 'Red', 'Green', 'Red', 'Green']  
['Red', 'Green', 'Green', 'Red', 'Green', 'Red', 'Green', 'Black', 'Green', 'Red']  
['Black', 'Red', 'Red', 'Red', 'Black', 'Red', 'Green', 'Black', 'Red', 'Red']  
['Red', 'Red', 'Black', 'Green', 'Green', 'Black', 'Green', 'Black', 'Red', 'Red']  
['Green', 'Black', 'Green', 'Red', 'Green', 'Green', 'Green', 'Green', 'Black', 'Green']  
['Green', 'Green', 'Green', 'Black', 'Black', 'Green', 'Black', 'Red', 'Red', 'Black']  
['Green', 'Black', 'Black', 'Green', 'Green', 'Black', 'Black', 'Red', 'Red', 'Red']
```

Simulation: Roulette

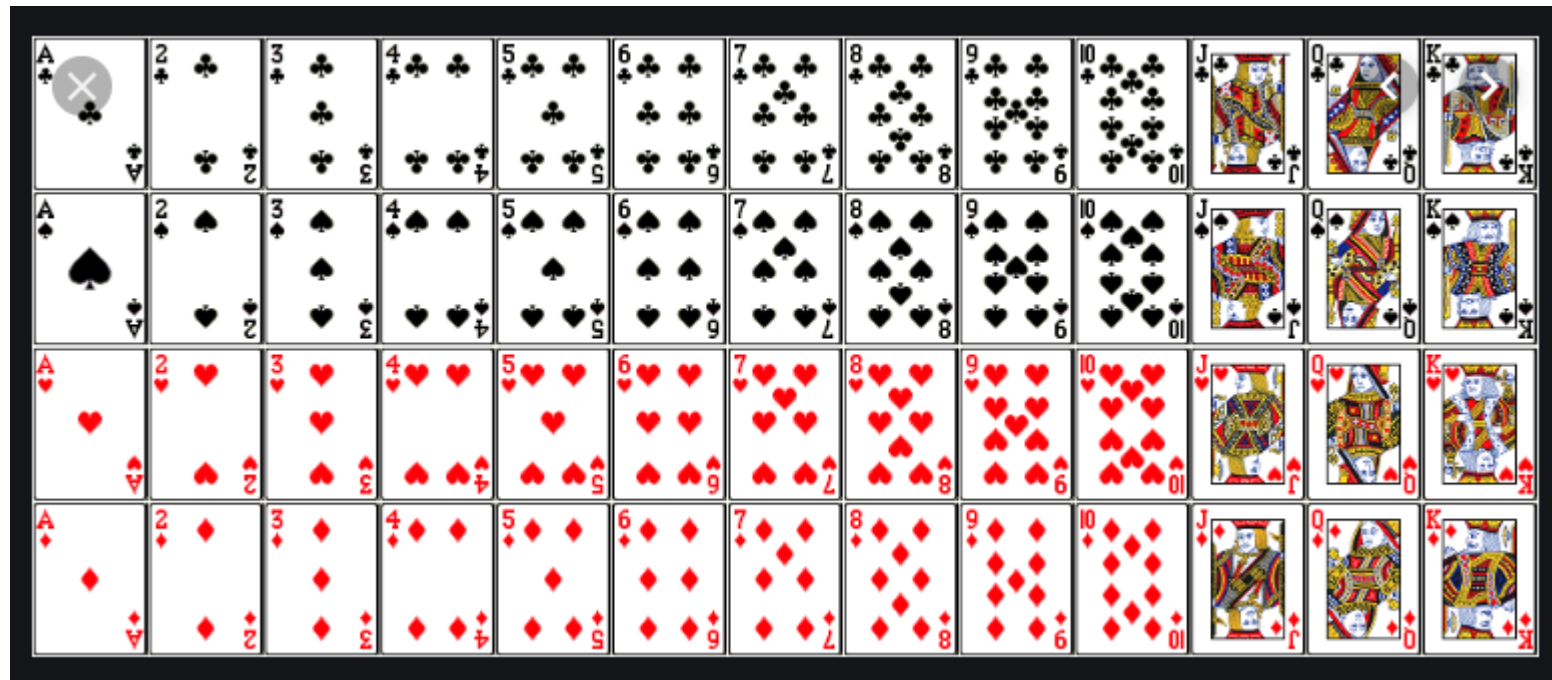
Weights: 18,18,2

```
In [16]: for i in range(10):  
         print(random.choices(colors, weights=[18,18,2], k = 10))
```

```
['Black', 'Red', 'Red', 'Red', 'Red', 'Black', 'Red', 'Black', 'Black', 'Black']  
['Black', 'Black', 'Red', 'Black', 'Black', 'Black', 'Red', 'Black', 'Red', 'Black']  
['Red', 'Black', 'Black', 'Black', 'Black', 'Red', 'Black', 'Red', 'Red', 'Red']  
['Black', 'Red', 'Black', 'Black', 'Black', 'Black', 'Black', 'Black', 'Red', 'Red']  
['Black', 'Black', 'Black', 'Black', 'Red', 'Black', 'Black', 'Black', 'Red', 'Black']  
['Black', 'Black', 'Red', 'Red', 'Black', 'Black', 'Red', 'Red', 'Black', 'Black']  
['Black', 'Black', 'Red', 'Red', 'Green', 'Red', 'Red', 'Green', 'Red', 'Red']  
['Red', 'Red', 'Green', 'Red', 'Black', 'Black', 'Red', 'Red', 'Red', 'Red']  
['Black', 'Black', 'Black', 'Red', 'Black', 'Black', 'Black', 'Black', 'Black', 'Green']  
['Black', 'Red', 'Black', 'Black', 'Red', 'Red', 'Red', 'Red', 'Red', 'Black']
```



Deck of Cards





Simulation: Deck of Cards

In [17]: *# Deck of Cards*

```
deck = list(range(1,53))  
print(deck)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52]
```

In [18]: `random.shuffle(deck)`

```
print(deck)
```

```
[5, 8, 42, 14, 38, 34, 39, 2, 17, 19, 24, 33, 51, 27, 48, 6, 49, 7, 31, 28, 40, 32, 16, 36, 3, 10, 46, 37, 30, 44, 47, 29, 1, 5, 0, 26, 13, 35, 22, 43, 12, 4, 20, 21, 25, 41, 52, 15, 11, 23, 18, 9, 45]
```

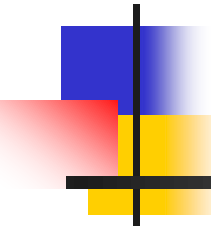
In [19]: `hand = random.sample(deck,k=5)`

```
print(hand)
```

```
[4, 37, 28, 8, 3]
```

In [20]: `suit = ['spade', 'club', 'heart', 'diamond']`
`num = ['1','2','3','4','5','6','7','8','9','10', 'jack', 'queen', 'king']`
`card_suit = random.sample(suit,1)`
`card_num = random.sample(num,1)`
`print(card_suit)`
`print(card_num)`

```
['spade']  
['king']
```



Sampling from Data Files

Data File

```
In [4]: top = Table.read_table('top_movies_2017.csv')
top
```

```
Out[4]:
```

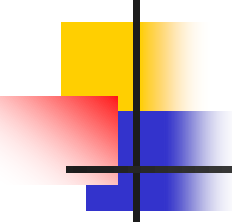
	Title	Studio	Gross	Gross (Adjusted)	Year
	Gone with the Wind	MGM	198676459	1796176700	1939
	Star Wars	Fox	460998007	1583483200	1977
	The Sound of Music	Fox	158671368	1266072700	1965
	E.T.: The Extra-Terrestrial	Universal	435110554	1261085000	1982
	Titanic	Paramount	658672302	1204368000	1997
	The Ten Commandments	Paramount	65500000	1164590000	1956
	Jaws	Universal	260000000	1138620700	1975
	Doctor Zhivago	MGM	111721910	1103564200	1965
	The Exorcist	Warner Brothers	232906145	983226600	1973
	Snow White and the Seven Dwarves	Disney	184925486	969010000	1937

... (190 rows omitted)

```
In [5]: top = top.with_column('Row Index', np.arange(top.num_rows)).move_to_start('Row Index')
top
```

```
Out[5]:
```

Row Index	Title	Studio	Gross	Gross (Adjusted)	Year
0	Gone with the Wind	MGM	198676459	1796176700	1939
1	Star Wars	Fox	460998007	1583483200	1977
2	The Sound of Music	Fox	158671368	1266072700	1965
3	E.T.: The Extra-Terrestrial	Universal	435110554	1261085000	1982
4	Titanic	Paramount	658672302	1204368000	1997
5	The Ten Commandments	Paramount	65500000	1164590000	1956
6	Jaws	Universal	260000000	1138620700	1975
7	Doctor Zhivago	MGM	111721910	1103564200	1965
8	The Exorcist	Warner Brothers	232906145	983226600	1973
9	Snow White and the Seven Dwarves	Disney	184925486	969010000	1937



Deterministic Sample

```
In [7]: #####  
# Deterministic Sample  
#  
top.take([3,5,7])
```

```
Out[7]:
```

Row Index	Title	Studio	Gross	Gross (Adjusted)	Year
3	E.T.: The Extra-Terrestrial	Universal	435,110,554	1,261,085,000	1982
5	The Ten Commandments	Paramount	65,500,000	1,164,590,000	1956
7	Doctor Zhivago	MGM	111,721,910	1,103,564,200	1965

Probabilistic Sample

```
In [9]: #####  
# Probabilistic Sample  
start = np.random.choice(np.arange(10))
```

```
In [10]: top.take(np.arange(start, start+5))
```

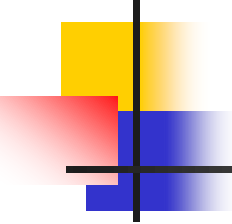
```
Out[10]:
```

Row Index	Title	Studio	Gross	Gross (Adjusted)	Year
4	Titanic	Paramount	658,672,302	1,204,368,000	1997
5	The Ten Commandments	Paramount	65,500,000	1,164,590,000	1956
6	Jaws	Universal	260,000,000	1,138,620,700	1975
7	Doctor Zhivago	MGM	111,721,910	1,103,564,200	1965
8	The Exorcist	Warner Brothers	232,906,145	983,226,600	1973

```
In [11]: #####  
# Uniform Random Sample  
#  
top.sample(5)
```

```
Out[11]:
```

Row Index	Title	Studio	Gross	Gross (Adjusted)	Year
165	Three Men and a Baby	Buena Vista	167,780,960	370,746,600	1987
116	Shrek	Dreamworks	267,665,011	420,413,700	2001
178	Funny Girl	Columbia	52,223,306	355,950,700	1968
50	Blazing Saddles	Warner Brothers	119,601,481	562,200,300	1974
23	Jurassic World	Universal	652,270,625	702,747,400	2015



Sampling + Group

```
In [12]: top.sample(50).group('Title')
```

```
Out[12]:
```

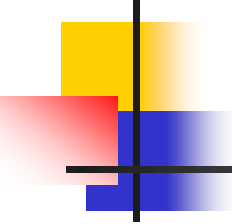
	Title	count
	101 Dalmatians	1
	Alice in Wonderland (2010)	2
	Armageddon	2
	Around the World in 80 Days	2
	Batman	1
	Batman v Superman: Dawn of Justice	1
	Beauty and the Beast (2017)	1
	Captain America: Civil War	1
	Cleopatra (1963)	1
	Dances with Wolves	1
... (35 rows omitted)		

Sampling with Replacement = False

```
In [13]: top.sample(5, with_replacement=False)
```

```
Out[13]:
```

Row Index	Title	Studio	Gross	Gross (Adjusted)	Year
2	The Sound of Music	Fox	158,671,368	1,266,072,700	1965
174	Austin Powers: The Spy Who Shagged Me	Newline	206,040,086	360,570,200	1999
21	The Graduate	AVCO	104,945,305	760,778,000	1967
177	Harry Potter and the Prisoner of Azkaban	Warner Brothers	249,541,069	357,233,500	2004
122	Crocodile Dundee	Paramount	174,803,506	410,739,900	1986



Summary

- Statistics
- Probability
 - Probability Simulation in Python
- Random Sampling
 - Sampling in Python